

WCS Implementation Pack

Operations manual + Swift Vapor starter sources

Version 1.0 · May 2026

World Class Scholars · Swift-first agentic operations

Executive operating model

World Class Scholars adopts a **Swift-first, supervised agentic platform**: bounded automation, explicit workflow state, policy-gated sensitive actions, and full audit trails. Agents coordinate through a central orchestrator; humans approve identity verification, certificate issuance, and external publishing.

Core principles (from case-study literature)

- 1. Coordination must be **constrained**, not merely automated.
- 2. Every agent has defined scope, fallback path, and audit trail.
- 3. Sensitive actions (identity, issuance, policy) remain gated until trust and performance thresholds are met.
- 4. Production systems use explicit state, bounded action spaces, and supervised transitions.

Swift-first architecture recommendation

Layer	Technology	Role
iOS client	SwiftUI + SwiftData	Participant & operator UX
Domain API	Vapor 4 + Fluent + async/await	System of record, workflows
Orchestration	Node/TypeScript (platform/)	Sessions, budgets, handoffs
Workers	Python (platform/agents/)	OCR, classification, reasoning
Policy	OPA/Rego	Pre/post invocation
Data	PostgreSQL + encrypted object store	Participants, audit, documents

Server-side Swift (Vapor) owns canonical business logic; the orchestrator owns session graphs and does not persist PII long-term.

Production readiness checklist

#	Item	Owner	Status
1	DPIA for onboarding + ID document	Legal	■
2	OPA policies in CI with deny tests	Engineering	■
3	Kill-switch tested in staging	Ops	■
4	Dual-control certificate issuance endpoint	OpenEng	■
5	Audit log retention policy (≥ 1 year)	Security	■
6	On-call runbook + incident playbook	Ops	■
7	SAST/DAST in release pipeline	DevSecOps	■
8	Token budgets per workflow configuration	Engineering	■
9	Human reviewer SLA defined (< 1 hr business)	Program	■
10	Pilot KPIs baselined (time-to-ready, precision)	Program	■
11	Red-team policy bypass exercise	Security	■
12	Rollback tested (model + policy budget)	Engineering	■

SaaS scaling roadmap

Phase	Scope	Deliverable
1	WCS pilot	Onboarding + approval UI
2	Supervised production	Certificate prep, support triage
3	Multi-tenant SaaS	Tenant isolation, KMS per tenant, config packs
4	Partner orgs	Self-serve sandbox, usage billing, SOC2 path

Generated: 2026-05-16

WCS Production Operating Manual

This manual turns enterprise agentic patterns (multi-agent coordination, equilibrium-style decisioning, phased maturity, security-by-design) into **production-ready World Class Scholars workflows** you can build, measure, and scale.

It extends the strategic playbook in docs/playbook/ with operational detail: case-study mapping, per-workflow control stacks, human gates, KPIs, and rollout stages.

Section	Document
Case-study → WCS operating model	01-case-study-operating-model.md
Universal control stack (every workflow)	02-universal-control-stack.md
Production architecture (layers)	03-production-architecture.md
Rollout maturity model	04-rollout-maturity-model.md
Workflow 1 — Participant onboarding	workflows/01-participant-onboarding.md
Workflow 2 — Learning-path concierge	[workflows/02-learning-path-concierge.md](workflows/02-learning-path-concierge.m
Workflow 3 — Certificate operations	workflows/03-certificate-operations.md
Workflow 4 — Research & campaign	[workflows/04-research-and-campaign.md](workflows/04-research-and-campaign.m

Runnable reference: platform/ (orchestrator + onboarding worker implemented; other workflows specified here for Phase 2 implementation).

Related: RACI & metrics · Governance · Architecture patterns

Case-study themes → WCS operating model

The source material converges on one principle: **selective autonomy**—agents where speed, repeatability, and coordination matter; humans at identity, issuance, and policy commits.

Theme map

Case-study pattern	What it means	WCS production use	Human control point
Independent learners	Each agent optimizes locally with separate agents: onboarding, CRIS, supervisor, content tagging, support triage	Onboarding and ID verification + certificate prep for initial assurance	Onboarding and ID verification + certificate prep for initial assurance
Joint-action learners	Agents coordinate with awareness of shared goals	Policy engine resolves competing interests	Escalation (e.g. confidentiality), shared data prep for
Consensus / CE / NE selection	Pick best joint action when options are limited	Block actions until consent, ID, profile completeness, data complete	Hard completeness, data complete
Constraint-based planning	Only feasible joint states execute	Pilot → supervised production → partner orgs	Stage gates that ease phase
Maturity model deployment	Basic → optimized stages		

Case-study pattern	What it means	WCS production use	Human control point
Security spectrum / EDR	Detect, respond, contain, learn	Monitor uploads, login anomalies, log4j and ms agentification signals	Incident management
SAST/DAST / shift-left	Security in pipeline	Scan code, APIs, dependencies, for security sigs	Security release

Pattern → architecture mapping

Pattern	WCS platform component
Independent learners	Dedicated **agent workers** + exception queue in governance UI
Joint-action learners	**Orchestrator** session graph with shared context store
Consensus / equilibrium	**Policy engine (OPA)** + conflict resolver in orchestrator
Constraint-based planning	Pre-invocation **Rego checks** + workflow state machine
Maturity model	Release gates G0–G3 ([04-rollout-maturity-model.md])(04-rollout-maturity-model.md)
Security / EDR	OpenTelemetry + audit store + kill-switch + anomaly rules
Shift-left	CI: SAST, dependency scan, secrets scan, policy bundle tests

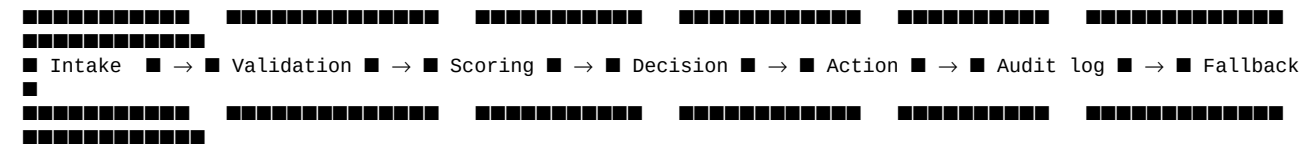
Four priority production workflows

#	Workflow	Pattern	Autonomy tier (initial)
1	[Participant onboarding](workflows/01-participant-agent-verify.md)	Self-anticipating Workflow	Supervised (human at ID + create)
2	[Learning-path concierge](workflows/02-learning-path-concierge.md)	Asynchronous Event Concierge	Partial (no PII writes)
3	[Certificate operations](workflows/03-certificate-operations.md)	Deterministic Peer Nodes	Supervised (no silent issuance)
4	[Research & campaign](workflows/04-research-campaign.md)	Agents Workflow - campaign	Supervised (publish blocked)

Implement in that order for ROI and risk: onboarding proves orchestration; certificate ops proves deterministic audit; concierge and research expand value with lower regulatory blast radius when scoped correctly.

Universal control stack (every WCS workflow)

Every production workflow uses the same seven stages. This mirrors coordination logic from multi-agent systems: bounded action spaces, explicit state, measurable outcomes.



Stage definitions

Stage	Responsibility	WCS implementation
Intake	Accept request, authenticate caller, attach context	API gate / orchestrator `POST /workflows/{type}/start`
Validation	Schema, consent, RBAC, data completeness	JSON schema + OPA `input.consent` + profile flags
Scoring	Risk, confidence, cost estimate	Worker returns `riskTier`, `confidence`; orchestrator enforces
Decision	Next node, approval required, or block	Policy engine + orchestrator state machine

Stage	Responsibility	WCS implementation
Action	Tool calls (email, CRM, LMS, storage)	Connectors with scoped credentials; idempotent where possible
Audit trail	Immutable append-only event per step	<code>`store.audit()`</code> → export to audit store / SIEM
Fallback	Retry, human queue, kill-switch, compensating action	Approval UI, <code>`WCS_KILL_SWITCH`</code> , session terminate

Pre-action checklist (orchestrator)

- Before any agent or tool invocation:
- 1. Session not killed; budget remaining
 - 2. OPA allow for (role, action, resource)
 - 3. Workflow state permits transition
 - 4. PII scope matches connector credential
 - 5. Loop count $\leq$ maxLoops (default 12)

Post-action checklist

- 1. Append audit event (actor, model version, tokens, tool calls)
- 2. Update session graph node status
- 3. If confidence $\leq$ threshold or risk $\geq$ medium → queue human approval
- 4. Emit metric (latency, success, escalation)

Conflict resolution (consensus layer)

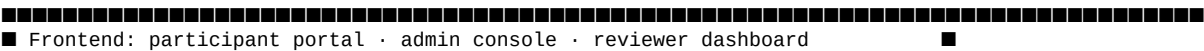
- When two workflows target the same participant:
- 1. Load participant lock / session mutex (orchestrator)
 - 2. Evaluate policies for priority: **safety > compliance > onboarding > engagement**
 - 3. If tie: queue decision to ops reviewer with both session IDs
 - 4. Never run certificate **issuance** while onboarding ID review is pending

Fallback matrix

Condition	Fallback
Worker timeout (3x retry)	Human queue + notify ops
Policy deny	Block + audit <code>`policy_denied`</code>
Budget exceeded	Pause session + approval to extend budget
Low confidence extraction	Human ID review (mandatory)
Runaway loop	Kill-switch + incident ticket template
Connector failure	Dead-letter queue + compensating email to participant

Production architecture (layered agent platform)

One giant autonomous agent is an anti-pattern. WCS production uses **layers** with clear contracts.



Component map (this repository)

Target SaaS stack (production-grade)

Agent responsibilities

Agent	Capabilities	Write scope
Onboarding	Email verify, ID extract, risk classify, CRM profile update	Platform draft only after approval
Certificate ops	Rule check, identity re-verify, issuance **payload** issuance commit	None
Learning-path concierge	Recommend modules, nudges, read LMS progress	Read-only; no PII export
Support triage	Classify ticket, suggest reply, route	CRM notes (supervised)
Content tagging	Tag assets, metadata	CMS write (low risk)
Analytics	Aggregates, cohort reports	Read-only warehouse

Agent	Capabilities	Write scope
Research & campaign	Summarize feedback, draft copy + citations	Drafts only until editorial approve

Service boundaries

- **Orchestrator** owns session lifecycle, budgets, handoffs, audit—not business persistence.
- **Vapor API** owns canonical participant/course/certificate records.
- **Workers** are stateless; all context passed in/out as versioned JSON schemas.
- **Policy engine** is sidecar; decisions versioned in CI with the deploy bundle.

Security operations alignment

Control	Maps to
Pre/post invocation hooks	NIST AC-3, AU-2
Kill-switch	IR containment
Token budgets	Resource availability / cost governance
Session replay	AU-9, forensic readiness
Approval gates	SoD for issuance and PII commits
CI policy tests	SA-11 shift-left

See governance.md for Rego examples and runtime controls.

Rollout maturity model

Phased deployment aligned with case-study maturity models: prove value under supervision before expanding autonomy.

Stages

Stage	Scope	Autonomy	Gate (G#)
0 — Design	Specs, policies, mock adversarial tests	None	G0
1 — Pilot	Onboarding only; human approval required	Low-risk steps automated	G0 → G1
2 — Supervised production	+ certificate prep, support triage	Sensitive actions gated	G1 → G2
3 — Multi-agent coordination	Shared sessions, conflict resolver	Partial autonomy on low-risk	G2
4 — Optimization	Dashboards, A/B policies, drift monitoring	Tunable per workflow	G2 sustained
5 — Scale-out	SaaS packaging for partner orgs	Tiered per tenant policy	G3

Stage 1 — Pilot (onboarding)

Duration: 4–8 weeks

Success criteria:

- Median time-to-ready –40% vs baseline
- ID precision ≥ 0.95, recall ≥ 0.90

- Zero critical policy violations
- 100% audit completeness on tool calls

Human capacity: Plan reviewer SLA (e.g., < 4h business hours for ID queue).

Stage 2 — Supervised production

Add:

- Certificate **preparation** (deterministic graph)
- Support triage agent (suggest-only until reviewer accepts)

Hard rule: certificate **issuance** remains dual-control; no LLM in commit path.

Stage 3 — Multi-agent coordination

- Orchestrator session mutex per participant
- Policy-based conflict resolution (02-universal-control-stack.md)
- Event bus for async handoffs (certificate prep triggered on onboarding complete)

Stage 4 — Optimization

- Per-workflow token budgets and cost dashboards
- Red-team quarterly on policy bypass and exfiltration paths
- Blue/green for worker model versions
- A/B policy bundles with automatic rollback on error-rate spike

Stage 5 — Scale-out (SaaS)

- Tenant isolation (DB schema or row-level + KMS per tenant)
- Configurable policy packs per jurisdiction
- Self-serve onboarding of new orgs with sandbox tier

Gate summary

Gate	Approvers	Evidence pack
G0 → Pilot	Ops + Engineering	MVP spec, PII DPIA, mock tests
G1 → Supervised	+ Legal/Security	30d pilot metrics, runbook
G2 → Partial autonomy	Governance board	Error budget, queue SLA, cost/workflow
G3 → Scale / SaaS	Board + Security	DR test, tenant isolation review, SOC2 path

Full RACI: raci-and-metrics.md.

Workflow 1 — Participant onboarding agent

Pattern: Sequenced Agent Workflow (joint-action with certificate prep handoff optional)

Autonomy tier: Supervised — human at ID review and participant record create

Reference implementation: platform/orchestrator/src/workflows/onboarding.js

Purpose

Collect profile fields, verify email, request passport/ID, classify risk, route edge cases to human reviewer, create participant record only after approval.

Actors

Role	RACI
Participant	Provides data and documents
Onboarding worker agent	Extracts, classifies risk
Orchestrator	Session, budget, sequencing
Reviewer	Approves/denies ID and profile
Vapor API	System of record for participants

Control stack

Stage	Actions
Intake	`POST /workflows/onboarding/start` with email, consent flags
Validation	Consent true; email format; document MIME allowlist
Scoring	Worker returns `riskTier`, `confidence`, `tokensUsed`
Decision	If risk=high OR confidence<0.85 → approval queue; else optional auto-path (policy)
Action	Email confirm (simulated/prod), ID extract, CRM draft, `POST /participants` after approval
Audit	`session_created`, `id_extract`, `approval_queued`, `workflow_completed`
Fallback	Worker retry 3x; then human queue; kill-switch pauses all sessions

Sequence

Participant → Orchestrator → [email_confirm] → Worker (ID extract) → Approval UI → Vapor (create) → Complete

Human control points

1. **Mandatory:** ID document review when riskTier != low OR confidence < 0.85
2. **Mandatory:** First participant create in production (policy require_human_create)
3. **Optional pilot:** WCS_AUTO_APPROVE=1 dev only

Data & connectors

System	Access
Object storage	Upload passport/ID (encrypted)
Email	Verification link
CRM	Draft contact (no promote without approval)
Vapor	Participant create

KPIs

Metric	Pilot target
Median time-to-ready	–40% vs baseline
ID precision / recall	≥ 0.95 / ≥ 0.90
% auto-completed (low-risk only)	$\leq 30\%$
Escalation to Tier 1	track; target $\leq 15\%$ at scale
Cost per onboarding	establish baseline

Policy hooks (OPA)

- `action == "extract_id" → role onboarding_worker, scope documents:read`
- `action == "create_participant" → requires approval.granted == true`
- Token budget: default 2000 tokens/session

Test plan

- Happy path with mock passport → approval → participant created
- Low-confidence doc → must queue approval
- Policy deny on create without approval → 403 + audit
- Kill-switch mid-session → session paused, no create

Phase 2 enhancements

- Real OCR model + liveness check vendor
- Mentor-match agent node after create
- CRM promote on approval

Workflow 2 — Learning-path concierge agent

Pattern: Autonomous Front-End (read-heavy) + scheduled nudge events

Autonomy tier: Partial — no unsupervised PII writes or enrollment commits

Purpose

Recommend modules, nudge unfinished tasks, personalize content by role, interests, and completion status.

Actors

Role	Responsibility
Participant	Consumes recommendations via portal/app
Concierge agent	Ranks modules, drafts nudges
LMS connector	Read progress, completions
Orchestrator	Optional: scheduled nudge sessions with token caps

Control stack

Stage	Actions
Intake	Authenticated participant session; `GET /recommendations` or chat turn
Validation	Participant ID matches token; no cross-tenant reads
Scoring	Relevance score + explanation string (for audit)
Decision	Cap recommendations at N=5; block if LMS unavailable
Action	Return ranked list; queue email/push only if opt-in
Audit	Log model version, sources (module IDs), no raw PII in prompts stored
Fallback	Static curriculum path if agent unavailable

Human control points

- **None** for read-only recommendations
- **Required** if recommending program **switch** or waiver (route to advisor queue)
- Editorial review if generated content shown publicly

Constraints (independent learner pattern)

- Separate worker from onboarding/certificate agents
- Supervisor reviews **exceptions** only (advisor queue)
- Hard token budget: 800 tokens/user/day (configurable)

KPIs

Metric	Target
Click-through on recommendations	+15% vs baseline
Module completion rate (cohort)	+10% at 90d
Nudge opt-out rate	< 5%
Hallucinated module references	**0** (validator checks LMS IDs)

Implementation notes

Component	Status
iOS Programs tab	UI surface (extend to call recommendations API)
Orchestrator route	`POST /workflows/concierge/start` — **spec only**
Worker	`platform/agents/concierge-worker/` — Phase 2
LMS connector	Read-only API key scoped to progress endpoints

Sequence (nudge batch)

Scheduler → Orchestrator → Concierge worker → LMS read → Policy allow → Send nudge (if opt-in) → Audit

Policy hooks

- Deny write:enrollment, write:grades
- Allow read:lms_progress for self only
- Block recommendations referencing unpublished modules

Workflow 3 — Certificate operations agent

Pattern: Deterministic Peer Nodes (BPMN-like; LLM not in commit path)

Autonomy tier: Supervised — **never issue without policy confirmation and approver**

Purpose

Check completion rules, validate identity consistency, build issuance payload, route to dual-control approval, record immutable issuance event.

Actors

Role	Responsibility
Certificate ops worker	Rule evaluation, payload assembly
Orchestrator	Deterministic state machine only
Approver A / B	Dual control (can be same role pool, different users)
Vapor API	Certificate record + hash

Control stack

Stage	Actions
Intake	Trigger: course completion event OR manual request
Validation	Participant `status=active`; ID verified; completion rules met
Scoring	N/A for LLM; rule engine returns pass/fail per criterion
Decision	All rules green → queue issuance approval; any fail → block with reason codes
Action	Build payload (participantId, courseId, templateId, hashes); **no** public cert until a
Audit	Immutable log: rule versions, approver IDs, payload hash, timestamp
Fallback	Manual certificate desk queue

Deterministic graph

[completion_event] → [rules_check] → [identity_reverify] → [payload_build] → [approval_A] → [approval_B] → [issuance_rec
 ■ fail[■■■■] → [blocked] + audit reason

Human control points

1. **Mandatory:** Dual approval before issuance_record
2. **Mandatory:** Identity re-verify if onboarding > 90 days ago
3. **Forbidden:** Auto-issuance via LLM tool call (policy enforced)

Joint-action coordination

- Mutex: cannot enter payload_build while onboarding session has id_review=pending
- On onboarding complete → eligible for rules_check (event-driven)

KPIs

Metric	Target
Preparation automation rate	≥ 80% payloads auto-built
Silent issuance incidents	**0** (release blocker)
Dual-control compliance	100% with approver ID in audit
Fraud/anomaly flags	100% investigated within SLA

Policy hooks (OPA)

```
# Illustrative – see platform/governance/policies/wcs.rego
deny[msg] {
  input.action == "issue_certificate"
  not input.approval.dual_control_complete
  msg := "dual control required"
}
```

Implementation status

Item	Status
Spec (this doc)	■
Orchestrator workflow	Phase 2 — `workflows/certificate.js`
Worker	Phase 2 — rule engine + template render
Vapor `certificates` endpoint	Extend implementation-pack

Test plan

- Completion without ID → blocked
- Single approval → deny issuance
- Tampered payload hash → detect on verify
- Concurrent onboarding → mutex blocks issuance

Workflow 4 — Research & campaign agent

Pattern: Sequenced Agent Workflow with **editorial approval** gate

Autonomy tier: Supervised — publish and external send blocked until human approve

Purpose

Draft program insights, summarize participant feedback (aggregated/de-identified where possible), produce campaign assets with **source citations**, route to editorial queue before publish or send.

Actors

Role	Responsibility
Research agent	Summarize, draft, cite sources
Analytics connector	Read aggregated feedback, survey exports
Editor	Approve copy, claims, and audience
Orchestrator	Workflow session + token budget

Control stack

Stage	Actions
Intake	Campaign brief (objective, audience, tone, deadline)
Validation	No raw PII in prompt context; sources must be allowlisted
Scoring	Citation coverage score; claim-risk flagger
Decision	High claim-risk → mandatory editorial; else draft to queue
Action	Write draft to CMS `status=draft` only; no email blast
Audit	Source doc IDs, model version, editor decision
Fallback	Human-only draft template

Human control points

1. **Mandatory:** External publish, email send, social post
2. **Mandatory:** Any statistic about participants (verify against warehouse)
3. **Recommended:** All public-facing copy in pilot

Citation requirements

Every factual bullet must include:

- `sourceId` (document, survey wave, or dashboard snapshot ID)
- `retrievedAt` timestamp
- `confidence` (high/medium/low)

Editor may reject insufficient citations.

KPIs

Metric	Target
Editor acceptance without major rewrite	≥ 60% (pilot)
Citation coverage on factual claims	100%
PII leakage incidents	**0**
Time brief-to-approved-draft	–30% vs manual

Sequence

Brief → Research worker (retrieve sources) → Draft + citations → Editorial approval → CMS publish / campaign send

Policy hooks

- Deny `send:email_external` without `editorial.approved`
- Deny prompts containing `participant.email`, `passport`, free-text PII fields
- Max tokens 4000/session; max 3 retrieval loops

Implementation status

Item	Status
Spec	■
Worker <code>`research-worker`</code>	Phase 2
Connectors	Google Drive read-only, warehouse read
Governance UI	Extend approval type <code>`editorial_review`</code>

Red-team scenarios

- Prompt injection via uploaded feedback doc
- Attempt to exfiltrate PII via “summarize all participants”
- Uncited statistical claim → must fail validation gate

Governance by design

Policy-as-code (templates)

Policies live in `platform/governance/policies/`. Deploy via CI; orchestrator loads bundle at startup.

Categories

1. **Data access** — which roles may read PII fields
2. **Allowed actions** — which tools each agent may invoke
3. **Budgets** — max tokens, max loops, max wall-clock per session
4. **Approvals** — which transitions require human approvedBy

Runtime controls

Control	Implementation
Kill-switch	<code>`POST /admin/kill-switch`</code> sets global flag; orchestrator rejects new runs
Token budget	Session counter; deny when exceeded
Approval gate	Workflow node <code>`type: human_approval`</code> blocks until UI action
Pre-action hook	OPA <code>`allow`</code> decision before tool call
Post-action audit	Append-only log per invocation

Audit trail (minimum fields)

- `sessionId`, `workflowId`, `nodeId`
- `agentId`, `modelId`, `modelVersion`

- toolName, inputHash, outputHash (not raw PII in logs)
- policyDecision, approverId (if applicable)
- timestamp, classification (public/internal/confidential)

NIST / ISO mapping (illustrative)

Principle	WCS control
Identify (NIST ID)	Asset inventory, data classification tags
Protect (PR)	RBAC, encryption at rest, TLS, policy-as-code
Detect (DE)	Monitoring alerts on policy denials, anomaly rate
Respond (RS)	Incident playbook, kill-switch, rollback
Recover (RC)	Workflow replay from audit, backup/restore drills
ISO A.12 Operations	Change management for policies/models
ISO A.18 Compliance	Retention schedule, DPIA for new agents

Observability

- Trace ID = sessionId across orchestrator, workers, Vapor API
- Metrics: workflow_started, workflow_completed, policy_denied, human_escalation
- Dashboards: Grafana/Datadog templates (hook via OTel exporter in orchestrator)

RACI & success metrics

RACI (program-level)

Activity	Program Director	Ops Lead	Engineering	Legal/Security	Faculty/Leadership
Process prioritization	A	R	C	C	I
Pattern selection	A	C	R	C	I
Policy definition	A	C	C	R	I
Pilot execution	I	R	R	C	I
Production gate	A	R	C	C	I
Incident response	I	R	R	C	I

A = Accountable · R = Responsible · C = Consulted · I = Informed

Decision gates

Gate	Criteria (all must pass)
G0 → Pilot	MVP spec signed; PII controls documented; mock adversarial tests green
G1 → Supervised production	Precision/recall ≥ targets; audit trail complete; on-call runbook
G2 → Partial autonomy	Error budget 30d green; human queue SLA met; cost/workflow within budget
G3 → Scale	Governance board sign-off; drift monitoring live; rollback tested

KPIs — participant onboarding (example)

Metric	Target (pilot)	Target (scale)
Time-to-ready (median)	−40% vs baseline	−55%
ID extraction precision	≥ 0.95	≥ 0.97
ID extraction recall	≥ 0.90	≥ 0.93
% completed without human approval (low-risk 80%)	sk 80%	≤ 50%
Escalation rate to Tier 1	tracked	< 15%
Cost per onboarding (tokens + infra)	<\$X	−20% YoY
Policy violation rate (blocked actions)	0 critical	0 critical

KPIs — certificate operations

Metric	Target
Preparation automation rate	≥ 80% payloads auto-built
Silent issuance incidents	**0** (hard gate)
Dual-control compliance	100% issuances with approver ID in audit log
Audit log completeness	100% tool calls + model version captured

Architecture patterns & selection

Pattern A — Sequenced Agent Workflow

Use when: Multi-step processes, shared state, validation between steps (onboarding, co-design).

[Orchestrator] → Agent A → validate → Agent B → validate → Human gate → Agent C → audit

- Orchestrator owns session, budgets, retries
- Each agent: single purpose, explicit I/O schema
- **WCS examples:** onboarding, campaign draft with editorial queue

Pattern B — Autonomous Front-End

Use when: Low-risk, exploratory queries; strict caps on tools and tokens.

- Hard token budget per session
- No access to write tools without policy allow
- Transactional handoff to human or workflow on ambiguity
- **WCS example:** campus assistant FAQ (pilot only)

Pattern C — Deterministic Peer Nodes

Use when: Regulated, auditable, BPMN-like flows (certificate issuance).

- Immutable audit trail per transition
- No LLM in commit path without human approval node

- **WCS example:** certificate preparation → approval → issuance record

Selection matrix

Criterion	Agent Workflow	Autonomous FE	Deterministic
Regulatory exposure	Medium–High	Low	High
Steps > 3	Yes	No	Yes
Needs LLM creativity	Yes	Yes	Rarely
Requires immutable audit	Yes	Partial	**Required**
Human approval	Per node	On escalation	**Mandatory** at commit

Core platform components (reference design)

Component	Responsibility
Orchestrator	Sessions, scheduling, budgets, handoffs, audit
Agent workers	Execute one capability; call tools via connectors
Tool connectors	Drive, email, CRM, LMS — scoped credentials
Policy engine	OPA/Rego pre/post hooks
Event bus	Async fan-out (optional at pilot)
Observability	OpenTelemetry, session replay
AuthN/Z	OIDC + RBAC
Data vault	Encrypted object store + KMS
Governance UI	Approvals, audit viewer

See runnable layout in `platform/README.md`.

Swift / Vapor implementation pack

Starter SaaS backend (Vapor 4, Fluent, PostgreSQL). Copy into implementation-pack/.

implementation-pack/Package.swift

```
// swift-tools-version: 6.0
import PackageDescription

let package = Package(
    name: "WCSAgenticSaaS",
    platforms: [
        .macOS(.v13),
    ],
    products: [
        .executable(name: "App", targets: ["App"]),
    ],
    dependencies: [
        .package(url: "https://github.com/vapor/vapor.git", from: "4.99.0"),
        .package(url: "https://github.com/vapor/fluent.git", from: "4.10.0"),
        .package(url: "https://github.com/vapor/fluent-postgres-driver.git", from: "2.11.0"),
        .package(url: "https://github.com/vapor/fluent-sqlite-driver.git", from: "4.0.0"),
        .package(url: "https://github.com/vapor/jwt.git", from: "4.2.0"),
    ],
    targets: [
        .executableTarget(
            name: "App",
            dependencies: [
                .product(name: "Vapor", package: "vapor"),
                .product(name: "Fluent", package: "fluent"),
                .product(name: "FluentPostgresDriver", package: "fluent-postgres-driver"),
                .product(name: "FluentSQLiteDriver", package: "fluent-sqlite-driver"),
                .product(name: "JWT", package: "jwt"),
            ]
        ),
        .testTarget(
            name: "AppTests",
            dependencies: [
                .target(name: "App"),
                .product(name: "XCTVapor", package: "vapor"),
            ]
        ),
    ]
)
```

implementation-pack/Dockerfile

```
FROM swift:6.0-jammy
WORKDIR /app
COPY . .
RUN swift build -c release
CMD [".build/release/App", "serve", "--hostname", "0.0.0.0", "--port", "8080"]
```

implementation-pack/docker-compose.yml

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: wcs
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d wcs"]
      interval: 3s
      timeout: 3s
      retries: 10

  app:
```

```

build: .
depends_on:
  db:
    condition: service_healthy
environment:
  DB_HOST: db
  DB_PORT: "5432"
  DB_USER: postgres
  DB_PASSWORD: postgres
  DB_NAME: wcs
  JWT_SECRET: change-me-in-production
ports:
  - "8080:8080"

```

implementation-pack/README.md

WCS Agentic SaaS Starter

Swift-first backend using Vapor for World Class Scholars.

Run locally

```

```bash
cd implementation-pack
docker compose up --build
```

```

Endpoints

```

- `POST /participants` - JSON `{ "email", "fullName" }`; creates participant and onboarding workflow run
- `POST /identity/upload` - JSON `{ "participantID", "documentURL" }`
- `POST /workflows/approve` - JSON `{ "workflowID", "approvedBy" }`
- `GET /health`

```

Native run (requires Postgres)

```

```bash
export DB_HOST=127.0.0.1
export DB_USER=postgres
export DB_PASSWORD=postgres
export DB_NAME=wcs
swift run App
```

```

Docker image

The image uses `swift:6.0-jammy` and starts the server with:

```
`App serve --hostname 0.0.0.0 --port 8080`
```

Note on `Package.swift` platforms

`platforms: [.macOS(.v13)]` sets the **minimum macOS** for Apple toolchains. If `docker compose build` fails on Linux with a platform error, remove the `platforms` line for Linux-only CI images.

implementation-pack/Sources/App/entrypoint.swift

```

import Logging
import Vapor

@main
struct Entrypoint {
    static func main() async throws {
        var env = try Environment.detect()
        try LoggingSystem.bootstrap(from: &env)

        let app = try await Application.make(env)
        do {
            try configure(app)
            try await app.autoMigrate()
            try await app.execute()
            try await app.asyncShutdown()
        } catch {
            try? await app.asyncShutdown()
            throw error
        }
    }
}

```

```

    }
  }
}

```

implementation-pack/Sources/App/configure.swift

```

import Fluent
import FluentPostgresDriver
import JWT
import Vapor

func configure(_ app: Application) throws {
    app.http.server.configuration.hostname = "0.0.0.0"
    app.http.server.configuration.port = Environment.get("PORT").flatMap { Int($0) } ?? 8080

    let dbHost = Environment.get("DB_HOST") ?? "localhost"
    let dbPort = Environment.get("DB_PORT").flatMap { Int($0) } ?? SQLPostgresConfiguration.ianaPortNumber
    let dbUser = Environment.get("DB_USER") ?? "postgres"
    let dbPassword = Environment.get("DB_PASSWORD") ?? "postgres"
    let dbName = Environment.get("DB_NAME") ?? "wcs"

    let sqlConfig = SQLPostgresConfiguration(
        hostname: dbHost,
        port: dbPort,
        username: dbUser,
        password: dbPassword,
        database: dbName,
        tls: .disable
    )

    app.databases.use(DatabaseConfigurationFactory.postgres(configuration: sqlConfig), as: .psql)

    app.migrations.add(CreateParticipantMigration())
    app.migrations.add(CreateWorkflowRunMigration())

    let jwtSecret = Environment.get("JWT_SECRET") ?? "development-only-change-me"
    app.jwt.signers.use(.hs256(key: jwtSecret), kid: "wcs")

    try routes(app)
}

```

implementation-pack/Sources/App/routes.swift

```

import Fluent
import Vapor

func routes(_ app: Application) throws {
    app.post("participants") { req async throws -> Participant in
        let input = try req.content.decode(CreateParticipant.self)
        let p = Participant(email: input.email, fullName: input.fullName)
        try await p.save(on: req.db)
        let run = WorkflowRun(
            workflowType: "onboarding",
            payload: "{\"participantID\":\"\\(p.id?.uuidString ?? \"\")\"}",
            riskScore: 0.1
        )
        try await run.save(on: req.db)
        return p
    }

    app.post("identity", "upload") { req async throws -> HTTPStatus in
        let input = try req.content.decode(UploadIdentityRequest.self)
        guard let participant = try await Participant.find(input.participantID, on: req.db) else {
            throw Abort(.notFound)
        }
        participant.status = input.documentURL.isEmpty ? "needs_review" : "identity_submitted"
        try await participant.save(on: req.db)
        return .accepted
    }

    app.post("workflows", "approve") { req async throws -> HTTPStatus in
        let input = try req.content.decode(ApproveRequest.self)
        guard let workflow = try await WorkflowRun.find(input.workflowID, on: req.db) else {
            throw Abort(.notFound)
        }
    }
}

```

```

        workflow.status = "approved"
        try await workflow.save(on: req.db)
        return .ok
    }

    app.get("health") { _ async in
        "ok"
    }
}

```

implementation-pack/Sources/App/WCSModels.swift

```

import Fluent
import Vapor

struct CreateParticipant: Content {
    let email: String
    let fullName: String
}

struct UploadIdentityRequest: Content {
    let participantID: UUID
    let documentURL: String
}

struct ApproveRequest: Content {
    let workflowID: UUID
    let approvedBy: String
}

final class Participant: Model, Content, @unchecked Sendable {
    static let schema = "participants"

    @ID(key: .id)
    var id: UUID?

    @Field(key: "email")
    var email: String

    @Field(key: "full_name")
    var fullName: String

    @Field(key: "status")
    var status: String

    @Timestamp(key: "created_at", on: .create)
    var createdAt: Date?

    init() {}

    init(id: UUID? = nil, email: String, fullName: String, status: String = "pending") {
        self.id = id
        self.email = email
        self.fullName = fullName
        self.status = status
    }
}

final class WorkflowRun: Model, Content, @unchecked Sendable {
    static let schema = "workflow_runs"

    @ID(key: .id)
    var id: UUID?

    @Field(key: "workflow_type")
    var workflowType: String

    @Field(key: "status")
    var status: String

    @Field(key: "payload")
    var payload: String

    @Field(key: "risk_score")
    var riskScore: Double

    @Timestamp(key: "created_at", on: .create)

```

```

    var createdAt: Date?

    init() {}

    init(workflowType: String, status: String = "queued", payload: String = "{}", riskScore: Double = 0) {
        self.workflowType = workflowType
        self.status = status
        self.payload = payload
        self.riskScore = riskScore
    }
}

```

implementation-pack/Sources/App/Migrations.swift

```

import Fluent

struct CreateParticipantMigration: AsyncMigration {
    func prepare(on db: Database) async throws {
        try await db.schema(Participant.schema)
            .id()
            .field("email", .string, .required)
            .field("full_name", .string, .required)
            .field("status", .string, .required)
            .field("created_at", .datetime)
            .unique(on: "email")
            .create()
    }

    func revert(on db: Database) async throws {
        try await db.schema(Participant.schema).delete()
    }
}

struct CreateWorkflowRunMigration: AsyncMigration {
    func prepare(on db: Database) async throws {
        try await db.schema(WorkflowRun.schema)
            .id()
            .field("workflow_type", .string, .required)
            .field("status", .string, .required)
            .field("payload", .string, .required)
            .field("risk_score", .double, .required)
            .field("created_at", .datetime)
            .create()
    }

    func revert(on db: Database) async throws {
        try await db.schema(WorkflowRun.schema).delete()
    }
}

```